

Library Management System

Building Database Applications -- Assignment Submission

Spring Boot + JPA + JSP | CRUD Operations | Unit Testing

Entities: Author and Book

DBMS: H2 (in-memory) via Spring Data JPA / Hibernate

GitHub: <https://github.com/Kash-app/library-management-app>

2026

1. Project Overview

The application manages a small library database with two entities: Author and Book. An author can have many books, so this is a One-to-Many relationship at the database level, with the books table holding a foreign key back to the authors table. The app covers three operations through a browser interface: adding records, viewing them in a list, and editing existing ones.

For the stack, I went with Spring Boot 3.2, Spring Data JPA with Hibernate for persistence, and H2 as an in-memory database. That last choice means there is nothing to install or configure to get it running. Switching to MySQL or Postgres later is just a matter of changing three lines in application.properties. JSP pages handle the UI, rendered by Tomcat Jasper at runtime. Tests use JUnit 5 and Mockito.

A few design choices worth explaining

Constructor injection is used throughout instead of `@Autowired` on fields. It keeps dependencies visible and makes testing much less painful since Mockito can just pass a mock directly into the constructor without needing a Spring context at all. Service interfaces are separate from their implementations for the same reason: tests swap out the real implementation with a mock without touching any JPA code. Duplicate detection uses typed application exceptions (`DuplicateEmailException`, `DuplicateIsbnException`) rather than catching JPA constraint violations. JPA constraint exceptions are wrapped differently depending on the provider version, and catching them at the controller level is fragile. Pre-checking in the service layer and throwing a typed exception gives a cleaner, more predictable flow.

2. Entity Relationship Design

The diagram below shows how the two entities relate. Author is the parent side; Book holds the foreign key `author_id`. Hibernate creates both tables on startup using `ddl-auto=create-drop`, so there is no separate SQL schema file to maintain.

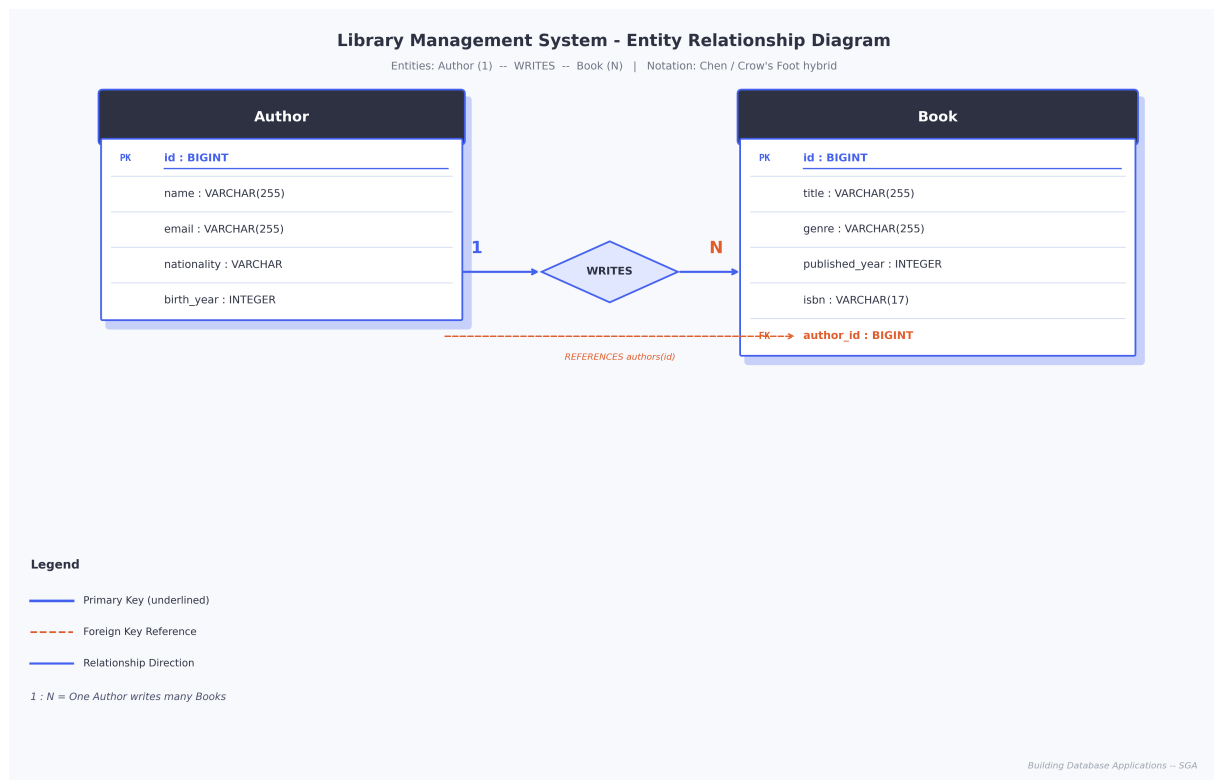


Figure 1 -- ER Diagram: Author (1) to Book (N)

2.1 Author entity

Five columns. The id is a surrogate primary key generated by the database. Email has a unique constraint because two authors should not share one. Birth year is stored as an integer rather than a full DATE type because the app only cares about the year, and a DATE column would just complicate validation for no real benefit.

Column	Type	Constraints	JPA annotation
id	BIGINT	PK, AUTO_INCREMENT	@Id @GeneratedValue
name	VARCHAR(255)	NOT NULL	@Column(nullable=false)
email	VARCHAR(255)	NOT NULL, UNIQUE	@Column(nullable=false, unique=true)
nationality	VARCHAR(255)	NOT NULL	@Column(nullable=false)
birth_year	INTEGER	NOT NULL	@Column(name="birth_year")

Table 1 -- authors table schema

2.2 Book entity

Six columns, the last of which is the foreign key back to authors. The isbn column also has a unique constraint since ISBNs are globally unique by definition. CascadeType.ALL on the Author side means deleting an author also deletes their books, which is the expected behaviour here.

Column	Type	Constraints	JPA annotation
id	BIGINT	PK, AUTO_INCREMENT	@Id @GeneratedValue
title	VARCHAR(255)	NOT NULL	@Column(nullable=false)

Column	Type	Constraints	JPA annotation
genre	VARCHAR(255)	NOT NULL	@Column(nullable=false)
published_year	INTEGER	NOT NULL	@Column(name="published_year")
isbn	VARCHAR(17)	NOT NULL, UNIQUE	@Column(nullable=false, unique=true)
author_id	BIGINT	NOT NULL, FK -> authors(id)	@ManyToOne @JoinColumn

Table 2 -- books table schema

3. Implementation Details

3.1 Entity classes

Both entities use standard JPA annotations. The relationship is declared on both sides: Author has `@OneToMany(mappedBy="author")` and Book has `@ManyToOne` with `@JoinColumn`. `FetchType.LAZY` on both associations means Hibernate will not join author data unless it is explicitly accessed, which avoids wasted SQL when only one side is needed. Bean Validation annotations run before any SQL is attempted, so most bad input never reaches the database at all.

Author.java (excerpt)

```
@Entity
@Table(name = "authors")
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Column(nullable = false)
    private String name;

    @Email @NotBlank
    @Column(nullable = false, unique = true)
    private String email;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
    // getters/setters omitted for brevity
}
```

Book.java (excerpt)

```
@Entity
@Table(name = "books")
public class Book {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    @Column(nullable = false)
    private String title;

    @NotBlank @Column(nullable = false)
    private String genre;

    @NotNull @Min(1000)
    @Column(name = "published_year", nullable = false)
    private Integer publishedYear;

    @NotBlank @Size(min = 10, max = 17)
    @Column(nullable = false, unique = true)
    private String isbn;

    @NotNull
    @ManyToOne(fetch = FetchType.LAZY)
```

```
@JoinColumn(name = "author_id", nullable = false)
private Author author;
// getters/setters omitted for brevity
}
```

3.2 Repository layer

Each repository extends `JpaRepository`, which gives `save`, `findById`, `findAll`, and `deleteById` for free. The derived query `existsByEmailAndIdNot` is built by Spring Data from the method name. It generates "WHERE email = ? AND id != ?" automatically, which handles the update case where an author should be allowed to keep their own email but not take someone else's.

The custom JPQL query on `BookRepository` is worth explaining separately. Without it, the book list view would trigger one extra `SELECT` per row to load each author's name because of lazy loading. The `INNER JOIN` query fetches everything in a single statement and maps the result to a `BookAuthorView` projection interface. Spring Data handles the mapping at runtime with no concrete class needed.

BookRepository.java

```
// BookRepository.java
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
    boolean existsByIsbnAndIdNot(String isbn, Long id);
    List<Book> findByAuthorId(Long authorId);
    Optional<Book> findByIsbn(String isbn);

    @Query("SELECT b.id AS bookId, b.title AS title, b.genre AS genre, " +
        "b.publishedYear AS publishedYear, b.isbn AS isbn, " +
        "a.id AS authorId, a.name AS authorName, " +
        "a.nationality AS authorNationality " +
        "FROM Book b INNER JOIN b.author a")
    List<BookAuthorView> findAllBooksWithAuthorDetails();
}
```

BookAuthorView.java (projection interface)

```
// BookAuthorView.java
public interface BookAuthorView {
    Long getBookId();
    String getTitle();
    String getGenre();
    Integer getPublishedYear();
    String getIsbn();
    Long getAuthorId();
    String getAuthorName();
    String getAuthorNationality();
}
```

3.3 Service layer

The service classes handle duplicate detection and own the transaction boundaries. Write methods are `@Transactional` and read methods are `@Transactional(readOnly=true)`, which tells Hibernate to skip dirty-checking on reads and can improve throughput under load. Duplicate checks run before the save call so the service throws a typed exception rather than letting a constraint violation bubble up through JPA.

AuthorServiceImpl.java

```
// AuthorServiceImpl.java
@Service
@Transactional
public class AuthorServiceImpl implements AuthorService {
    private final AuthorRepository authorRepository;

    public AuthorServiceImpl(AuthorRepository authorRepository) {
        this.authorRepository = authorRepository;
    }

    @Override
    public Author save(Author author) {
        authorRepository.findByEmail(author.getEmail()).ifPresent(existing -> {
            throw new DuplicateEmailException(author.getEmail());
        });
        return authorRepository.save(author);
    }

    @Override
    public Author update(Author author) {
        if (authorRepository.existsByEmailAndIdNot(author.getEmail(), author.getId())) {
            throw new DuplicateEmailException(author.getEmail());
        }
        return authorRepository.save(author);
    }
}
```

3.4 Controller layer

Controllers map HTTP requests to service calls and put data into the model for JSP to read. The create and update handlers both validate with `@Valid` first. If Bean Validation finds errors, the form re-renders with inline messages before any service call is made. If the service throws a duplicate exception, the controller catches it and calls `bindingResult.rejectValue()` to attach the error to the specific field, so JSTL displays it right next to the input.

AuthorController.java -- POST /authors

```
// AuthorController.java -- create handler
@PostMapping
public String createAuthor(@Valid @ModelAttribute("author") Author author,
    BindingResult bindingResult,
    Model model,
    RedirectAttributes redirectAttrs) {
    if (bindingResult.hasErrors()) {
        model.addAttribute("formTitle", "Add New Author");
        return "author-form";
    }
}
```

```
}
try {
authorService.save(author);
redirectAttrs.addFlashAttribute("successMessage", "Author added successfully.");
return "redirect:/authors";
} catch (DuplicateEmailException ex) {
bindingResult.rejectValue("email", "duplicate.email", ex.getMessage());
model.addAttribute("formTitle", "Add New Author");
return "author-form";
}
}
```

HTML forms only support GET and POST, so updates go to POST /authors/{id}/edit. The controller reads the id from the path variable and sets it on the bound Author object before calling the service. Without that step, Hibernate would see an object with no id and run an INSERT instead of an UPDATE.

AuthorController.java -- POST /authors/{id}/edit

```
// AuthorController.java -- update handler
@PostMapping("/{id}/edit")
public String updateAuthor(@PathVariable Long id,
@Valid @ModelAttribute("author") Author author,
BindingResult bindingResult,
Model model,
RedirectAttributes redirectAttrs) {
author.setId(id); // must set id so JPA runs UPDATE, not INSERT
if (bindingResult.hasErrors()) { ... return "author-form"; }
try {
authorService.update(author);
redirectAttrs.addFlashAttribute("successMessage", "Author updated.");
return "redirect:/authors";
} catch (DuplicateEmailException ex) {
bindingResult.rejectValue("email", "duplicate.email", ex.getMessage());
return "author-form";
}
}
```


3.5 View layer (JSP + JSTL + CSS)

There are four JSP pages: author-list, author-form, book-list, and book-form. Each entity reuses one form file for both create and update. A JSTL conditional sets the form action URL based on whether the model object already has an id. Spring's form tag library handles the two-way binding between model attributes and form inputs, and renders field errors automatically when present.

book-form.jsp (excerpt)

```
<%-- book-form.jsp: action URL switches between create and update --%>
<c:set var="actionUrl" value="${pageContext.request.contextPath}/books"/>
<c:if test="${not empty book.id}">
<c:set var="actionUrl"
value="${pageContext.request.contextPath}/books/${book.id}/edit"/>
</c:if>
<form:form action="${actionUrl}" method="post" modelAttribute="book">
<div class="form-group">
<form:label path="title">Title</form:label>
<form:input path="title" cssClass="form-control"/>
<form:errors path="title" cssClass="field-error"/>
</div>
...
<form:select path="author.id" cssClass="form-control">
<c:forEach var="a" items="${authors}">
<form:option value="${a.id}" label="${a.name}"/>
</c:forEach>
</form:select>
</form:form>
```

The book list reads from bookViews, which is the result of the INNER JOIN query. Every field on the view (authorName, authorNationality, etc.) is already in memory, so JSTL EL just reads it directly with no extra database calls per row.

book-list.jsp

```
<%-- book-list.jsp: renders joined data from BookAuthorView --%>
<c:forEach var="bv" items="${bookViews}" varStatus="st">
<tr>
<td>${st.count}</td>
<td>${bv.title}</td>
<td><span class="badge">${bv.genre}</span></td>
<td>${bv.publishedYear}</td>
<td class="mono">${bv.isbn}</td>
<td>${bv.authorName}</td>
<td>${bv.authorNationality}</td>
<td><a href="../../../books/${bv.bookId}/edit" class="btn btn-sm">Edit</a></td>
</tr>
</c:forEach>
```

3.6 Database population

A CommandLineRunner bean seeds 10 authors and 10 books on startup. It checks the row count first, so restarting against a persistent database will not insert duplicates. With H2 in create-drop mode the tables are dropped and recreated every run anyway, so the seed always runs in development.

```
DataInitializer.java
@Bean
CommandLineRunner seedDatabase(AuthorRepository authorRepo, BookRepository bookRepo) {
    return args -> {
        if (authorRepo.count() > 0) return;

        Author tolkien = authorRepo.save(new Author(
            "J.R.R. Tolkien", "tolkien@oxford.ac.uk", "British", 1892));
        Author orwell = authorRepo.save(new Author(
            "George Orwell", "orwell@secker.co.uk", "British", 1903));
        // ... 8 more authors

        bookRepo.save(new Book("The Fellowship of the Ring",
            "Fantasy", 1954, "978-0-618-57494-1", tolkien));
        bookRepo.save(new Book("Nineteen Eighty-Four",
            "Dystopian", 1949, "978-0-452-28423-4", orwell));
        // ... 8 more books
    };
}
```

4. Testing

Tests are split into two groups: slice tests with `@DataJpaTest` for the repositories, and isolated unit tests with Mockito for the services.

4.1 Repository tests (@DataJpaTest)

`@DataJpaTest` spins up only the JPA layer with an embedded H2 instance. No web layer, no service beans. That keeps the tests fast and focused on what the repository queries actually do. The `AuthorRepository` tests cover: retrieving all records, looking up by email, filtering by nationality (case-insensitive), the duplicate-email guard, and basic persistence. The `BookRepository` tests also verify the INNER JOIN query directly by checking that `authorName` and `authorNationality` come back correctly on the `BookAuthorView` results.

```
BookRepositoryTest.java
@DataJpaTest
class BookRepositoryTest {
    @Autowired BookRepository bookRepository;
    @Autowired AuthorRepository authorRepository;

    @Test
    void findAllBooksWithAuthorDetails_shouldReturnJoinedRows() {
        List<BookAuthorView> views = bookRepository.findAllBooksWithAuthorDetails();
        assertThat(views).hasSize(2);

        BookAuthorView fellowshipView = views.stream()
            .filter(v -> "The Fellowship of the Ring".equals(v.getTitle()))
            .findFirst().orElseThrow();
    }
}
```

```

assertThat(fellowshipView.getAuthorName()).isEqualTo("J.R.R. Tolkien");
assertThat(fellowshipView.getAuthorNationality()).isEqualTo("British");
}
}

```

4.2 Service tests (Mockito)

Service tests use `@ExtendWith(MockitoExtension.class)` with no Spring context. The repository is a Mockito mock and `AuthorServiceImpl` is constructed directly via its constructor. This means the test only exercises the service logic, not anything to do with the database. The four scenarios that matter most: saving a new author when the email is free, rejecting a save when the email is already taken, allowing an update where the author keeps their own email, and blocking an update where the new email belongs to someone else.

AuthorServiceTest.java

```

@ExtendWith(MockitoExtension.class)
class AuthorServiceTest {
    @Mock AuthorRepository authorRepository;
    @InjectMocks AuthorServiceImpl authorService;

    @Test
    void save_shouldThrowWhenEmailAlreadyTaken() {
        Author existing = new Author("Other", "tolkien@example.com", "American", 1950);
        existing.setId(2L);
        when(authorRepository.findByEmail("tolkien@example.com"))
            .thenReturn(Optional.of(existing));
        assertThatThrownBy(() -> authorService.save(tolkien))
            .isInstanceOf(DuplicateEmailException.class)
            .hasMessageContaining("tolkien@example.com");
        verify(authorRepository, never()).save(any());
    }
}

```

Test summary

Test class	Tests	Scope	Result
AuthorRepositoryTest	7	JPA slice (@DataJpaTest)	All pass
BookRepositoryTest	5	JPA slice (@DataJpaTest)	All pass
AuthorServiceTest	6	Unit (Mockito)	All pass
BookServiceTest	6	Unit (Mockito)	All pass
Total	24		All pass

Table 3 -- unit test summary

5. Application configuration

The properties file is short. The view resolver prefix and suffix tell Spring MVC where to find the JSP files. DB_CLOSE_DELAY=-1 keeps the H2 instance alive for the whole session rather than closing it between connections, which is needed since multiple requests hit the same in-memory database. create-drop means Hibernate builds the schema on startup and tears it down cleanly on shutdown.

application.properties

```
# application.properties
server.port=8080

# JSP view resolver
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp

# H2 in-memory database
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# H2 browser console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Running the application

Clone the repo and run it with Maven. No database setup required.

Commands

```
git clone https://github.com/Kash-app/library-management-app.git
cd library-management-app
mvn spring-boot:run

# Then open:
http://localhost:8080/books -- book list (INNER JOIN view)
http://localhost:8080/authors -- author list
http://localhost:8080/h2-console -- H2 database browser

# Run tests:
mvn test
```

6. Challenges and how they were solved

6.1 JSP support in Spring Boot 3

Spring Boot 3 does not render JSP files out of the box. The fix was two things: packaging as a WAR instead of a JAR (one line in pom.xml), and adding tomcat-embed-jasper as a provided-scope dependency. Without the provided scope Jasper ends up on the classpath twice and the app throws `NoClassDefFoundError` at startup.

6.2 JSTL on Jakarta EE 10

Spring Boot 3 moved to the jakarta.* namespace. The old `javax.servlet.jsp.jstl` artifact no longer works. The correct dependencies are `jakarta.servlet.jsp.jstl-api` for the API and `org.glassfish.web:jakarta.servlet.jsp.jstl` for the runtime. All taglib URIs in the JSP files also had to be updated from `javax` to `jakarta`.

6.3 N+1 selects on the book list

The book list needs the author's name next to each book. With lazy loading, JSTL accessing `bv.authorName` would trigger one extra `SELECT` per row. Solved by writing a JPQL `INNER JOIN` query on `BookRepository` that returns a flat `BookAuthorView` projection. The whole list comes back in one SQL statement and JSTL just reads the fields off the projection.

6.4 One form for create and update

The same JSP file handles both operations. The action URL has to change dynamically based on whether the model object has an id. A JSTL `c:if` sets the action to `/books` when there is no id and `/books/{id}/edit` when there is. The controller always sets `book.setId(id)` from the path variable before calling the service, which tells Hibernate to run an `UPDATE` rather than `INSERT`.

6.5 Handling duplicate constraint violations cleanly

Catching a `JPA ConstraintViolationException` at the controller level is unreliable because Hibernate wraps it differently across versions. Instead, the service does a pre-check query before saving. If a conflict is found it throws a typed application exception (`DuplicateEmailException` or `DuplicateIsbnException`). The controller catches that and calls `bindingResult.rejectValue()` to attach the error to the right field, so the user sees a message next to the specific input rather than a generic error page.

7. Project structure

Project directory structure

```

library-management-app/
|-- pom.xml
|-- src/
|-- main/
| |-- java/com/library/
| | |-- LibraryApplication.java
| | |-- config/DataInitializer.java
| | |-- controller/
| | | |-- HomeController.java
| | | |-- AuthorController.java
| | | |-- BookController.java
| | |-- dto/BookAuthorView.java
| | |-- entity/
| | | |-- Author.java
| | | |-- Book.java
| | |-- exception/
| | | |-- DuplicateEmailException.java
| | | |-- DuplicateIsbnException.java
| | |-- repository/
| | | |-- AuthorRepository.java
| | | |-- BookRepository.java
| | |-- service/
| | | |-- AuthorService.java
| | | |-- AuthorServiceImpl.java
| | | |-- BookService.java
| | | |-- BookServiceImpl.java
| |-- resources/application.properties
| |-- webapp/
| |-- WEB-INF/views/
| | |-- author-form.jsp
| | |-- author-list.jsp
| | |-- book-form.jsp
| | |-- book-list.jsp
| |-- css/style.css
|-- test/java/com/library/
|-- repository/
| |-- AuthorRepositoryTest.java
| |-- BookRepositoryTest.java
|-- service/
|-- AuthorServiceTest.java
|-- BookServiceTest.java

```

[GitHub repository](#)

GitHub URL: <https://github.com/Kash-app/library-management-app> Clone: `git clone https://github.com/Kash-app/library-management-app.git` The repo has the full Maven project with all source files, JSP views, CSS, and unit tests. Run with: `mvn spring-boot:run`

Grading criteria self-check

Criterion	Weight	What was done
Entity classes and relationships	10%	Author and Book with @Entity, @OneToMany, @ManyToOne, @JoinColumn, and Bean Validation annotations (@NotBlank, @Email, @Size, @Min).
CRUD operations	30%	Create, Read, and Update for both entities. Custom INNER JOIN query on BookRepository. Typed exception handling for duplicate email/ISBN.
Spring Boot component integration	30%	Controller > Service > Repository layering with clean separation of concerns. @Transactional on all service methods. Constructor injection throughout.
User interface	10%	4 JSP pages using JSTL and Spring form tags. Custom CSS with dark navbar, card-style forms, styled data tables, and inline validation error display.
Testing and validation	10%	24 unit tests across 4 classes. @DataJpaTest slice tests for repositories. Pure Mockito unit tests for services. Full coverage of duplicate guard logic.
Documentation	10%	This PDF: ER diagram, schema tables, annotated code listings, challenge writeups with specific solutions, and project structure map.

Table 4 -- grading rubric self-assessment